Scan Worker Deployment

Use this worker when the control plane is hosted on Vercel and the heavy scan execution needs to run somewhere with longer process lifetime and access to the scanner binaries.

Purpose

The worker runs:

- multi-cloud scans
- direct schedule execution
- recurring schedule processing
- scanner binaries that do not belong inside Vercel Functions

Recommended targets

- Render web service + cron job
- Fly.io app + machine/cron setup
- Railway service
- ECS/Fargate
- a small VM with Docker

Start command

```
WEB_CONCURRENCY=${WEB_CONCURRENCY:-2} \
uvicorn backend.worker_app:app --host 0.0.0.0 --port ${PORT:-8010} --workers ${WEB_CONCU
```

Using at least two worker processes is recommended for this project because a long-running scan can otherwise block `/health` checks and make the control plane think the worker is offline.

This repo also includes host-specific launch helpers:

- `deploy/run-worker.sh`
- `deploy/run-due-schedules.sh`

Required environment variables

- `DATABASE_URL`
- `WORKER_API_TOKEN`
- `SECRET_KEY`
- `ENCRYPTION_KEY`

Common optional variables:

- `OPENAI_API_KEY`
- `OPENAI_AGENT_MODEL`
- `AWS_REGION`

- AWS_PROFILE
- AWS_ACCESS_KEY_ID
- AWS_SECRET_ACCESS_KEY
- GCP_PROJECT_ID
- GCP_SERVICE_ACCOUNT_JSON
- SMTP_HOST
- SMTP_PORT
- SMTP_USER
- SMTP_PASSWORD
- SMTP_FROM_EMAIL

EKS-based Kubernetes scanning

If you save an Amazon EKS kubeconfig generated by `aws eks update-kubeconfig`, the worker must be able to execute `aws eks get-token`. For that setup:

- install `awscli` on the worker host
- ensure the worker has outbound access to the EKS API endpoint
- save a valid AWS credential in CloudGuard; the worker injects that credential into the kubeconfig exec environment for EKS token generation
- paste a portable kubeconfig into CloudGuard, for example from:

```
kubectl config view --raw --minify --flatten
```

Control-plane variables on Vercel

Set these on the Vercel project so the web app forwards heavy work:

- SCAN_WORKER_URL=https://your-worker.example.com
- SCAN_WORKER_TOKEN=the-same-value-as-WORKER_API_TOKEN

Render config in this repo

- render.worker.yaml

It defines:

- one public web service for backend.worker_app:app
- one cron job that runs backend.run_due_schedules_once every 5 minutes

Import the Blueprint in Render, then fill in the prompted secrets.

Railway config in this repo

- deploy/railway.worker.toml
- deploy/railway.cron.toml

Railway config is per service. Create two services from the same repo:

1. Worker API service Set the config file path to `/deploy/railway.worker.toml`
2. Cron service Set the config file path to `/deploy/railway.cron.toml`

Both services should receive the same runtime environment values except that only the worker API service needs `WORKER_API_TOKEN`.

Health check

```
curl https://your-worker.example.com/health
```

Running due schedules without a long-lived loop

If your host does not support a persistent worker loop, call this endpoint from the host's cron mechanism:

```
curl -X POST \
  -H "Authorization: Bearer $WORKER_API_TOKEN" \
  https://your-worker.example.com/internal/schedules/run-due
```